FLUTTER • DART • OFFLINE AUDIO

PREMIUM MUSIC PLAYER

Your local music library, playlists, and playback in one app

PROJECT DOCUMENTATION

This document outlines the purpose, features, application structure, technology stack, and possible future improvements of the Premium Music Player.

1. Introduction

Premium Music Player is a Flutter-based mobile application designed to play music stored locally on a user's device. It focuses on a clean listening experience, personal playlists, and convenient playback controls without requiring a user account or a server.

The app is intended for people who want to organize and listen to their downloaded songs in one place. By relying on local storage for playlists and preferences, it aims to keep the core listening experience available without an online music service.

Project overview

Category	Details
Project name	Premium Music Player
Application type	Local music player / mobile application
Framework	Flutter
Programming language	Dart
Storage approach	Local device storage
Primary goal	Play local audio and manage personal playlists

2. Problem Statement and Objectives

Users who keep music files on their phones need a convenient way to browse songs, play tracks, and organize their own collections. A music app that depends on online accounts or streaming services may not fit users who primarily listen to downloaded audio.

Premium Music Player addresses this use case by focusing on locally stored songs and playback features in a dedicated mobile interface.

Objectives

- Play music files available on the user's device.
- Provide a simple interface for browsing a local music library.
- Allow users to create and manage custom playlists.
- Support background playback and convenient playback controls.
- Provide lock-screen media controls where supported by the device and operating system.
- Store playlists and app data locally without requiring a server or account.
- Create a polished and responsive mobile user experience.

3. Main Features

Local music library

Access and browse supported audio files stored on the device, subject to the permissions granted by the user.

Music playback

Play selected tracks and provide standard playback actions such as play, pause, next, and previous.

Custom playlists

Create personal playlists from songs in the local library and keep playlist information in local storage.

Background playback

Continue audio playback when the user leaves the app, when supported by the playback service and platform configuration.

Lock-screen controls

Expose media controls on the lock screen and system media interface where supported and correctly configured.

Offline-first design

Core music playback and locally stored playlists do not require a music streaming server or user account.

Player interface

Provide a dedicated playback screen for the current track and its controls.

Local preferences

Use on-device storage for app data such as playlists and other supported preferences.

4. User Interface and User Flow

The application is intended to keep music browsing and playback straightforward. Users should be able to find a track, start playback, and return to their music collection without unnecessary steps.

Typical user flow

Step	User action	App response
1	Opens the app	Loads the music interface and available local library.
2	Browses songs	Displays the available tracks discovered by the app.
3	Selects a track	Starts playback and opens or updates the player screen.
4	Creates a playlist	Lets the user name a playlist and add selected songs.
5	Leaves the app	Playback can continue in the background when configured and supported.
6	Locks the phone	Media controls can be available through the system lock screen.

User experience considerations

- Keep play, pause, next, and previous controls easy to identify.
- Show useful song information such as title and artist when available in file metadata.
- Make playlist creation and selection clear.
- Request audio permissions only when needed and explain why access is required.
- Handle empty libraries and permission-denied states with helpful messages.

5. Technology Stack

The application is built with Flutter and Dart. Local audio discovery, playback, and persistence can be handled through Flutter packages and device APIs. The following table summarizes the technologies associated with the project implementation.

Technology / package	Role
Flutter	Builds the mobile user interface from a shared codebase.
Dart	Implements app logic, state handling, and user interactions.
on_audio_query	Can query local audio files and their metadata on supported Android devices.
audio_service	Supports background audio tasks and integration with system media controls when configured.
Hive	Local key-value storage used for playlists and app data in the described implementation.
Riverpod	State-management approach used to organize and share application state.

Package behavior depends on version, platform permissions, and app configuration. Features such as background playback and lock-screen controls should be tested on the target Android device.

6. Application Structure

The project separates the user interface, playback behavior, local library access, and saved app data. This separation helps keep the app easier to maintain and extend.

```
PremiumMusicPlayer/
■■■ lib/
■ ■■■ main.dart
■ ■■■ screens/
■ ■■■ widgets/
■ ■■■ services/
■ ■■■ state/
■■■ android/
■■■ assets/ (if used)
■■■ pubspec.yaml
■■■ README.md
```

This is a suggested high-level structure. Actual folder and file names may differ in the current project.

Core components

Component	Responsibility
Library screen	Lists or searches music tracks available on the device.
Player screen	Displays the current track and playback controls.
Audio service	Coordinates playback and background/system media integration.
Playlist storage	Saves playlist names and associated track information locally.
Application state	Keeps screen data and playback-related UI state coordinated.

7. Local Storage, Privacy, and Permissions

The app is designed around local music files and local app data. It does not require a dedicated server or user account for its core use case.

Local storage

- Playlist information can be saved on the device.
- App preferences and supported user-created data can remain local.
- The actual music files remain in the device's existing storage unless the app explicitly copies or manages them.

Permissions and privacy

- Android may require media/audio permissions before the app can read the user's music library.
- Permission handling should account for differences between Android versions.
- The app should clearly communicate why music access is requested.
- No server or account is required for the described offline listening workflow.

Privacy note

Local-first design reduces dependence on online services, but the app's actual data behavior depends on its implemented packages and code. Any analytics, backups, or network access should be documented accurately if added.

8. Testing and Quality Checks

Testing should cover both ordinary playback and the platform behaviors needed for background audio. Use a physical Android device where possible, as emulator and device behavior may differ.

- Verify that the app requests and handles media permissions correctly.
- Check that supported local audio files appear in the library.
- Test play, pause, next, previous, and track selection.
- Create a playlist, add songs, close the app, and verify that the playlist persists.
- Test playback after switching to another app.
- Test lock-screen and notification media controls.
- Check behavior when the music library is empty or permissions are denied.
- Test audio interruption behavior, such as calls or other audio apps.
- Check performance, screen responsiveness, and error handling on the target device.

9. Future Enhancements

The following improvements could extend the player beyond its current core scope. These are possible future additions, not claims that they are already implemented.

Search and filters

Search by song title, artist, album, or other available metadata.

Favorites

Let users mark tracks as favorites for quick access.

Playlist editing

Support renaming playlists, reordering tracks, and removing songs.

Equalizer and audio settings

Offer playback customization if supported by the target platform.

Themes

Provide light, dark, and accent-color options.

Sleep timer

Stop playback automatically after a selected duration.

Queue management

Allow users to view and reorder upcoming tracks.

Backup and restore

Offer an optional export/import method for playlist metadata without requiring a streaming account.

10. Conclusion

Premium Music Player is a Flutter-based local music player designed to bring downloaded music, custom playlists, and convenient playback controls together in one application. Its offline-first approach avoids the need for a streaming server or user account for core listening.

The project demonstrates mobile app development using Flutter and Dart, along with local persistence and audio-related packages. Continued work can focus on reliable background playback, lock-screen integration, playlist usability, permissions, and performance across Android devices.

Project summary

Category	Summary
Project name	Premium Music Player
Application type	Offline local music player
Framework / language	Flutter / Dart
Storage	Local device storage
Key features	Local library, playback, playlists, background audio, lock-screen controls
Account / server	Not required for core offline listening
Developer	Sai Pavan
Program / institution	B.Tech — Computer Science and Engineering, VIT-AP University

PREMIUM MUSIC PLAYER Your music library. Your playlists. Your listening experience.